

LAPORAN PRAKTIKUM SISTEM OPERASI

MODUL 7 SEMAPHORE



Disusun Oleh :

Ilham Lii Assidaq 2311104068

Dosen Pengampu :

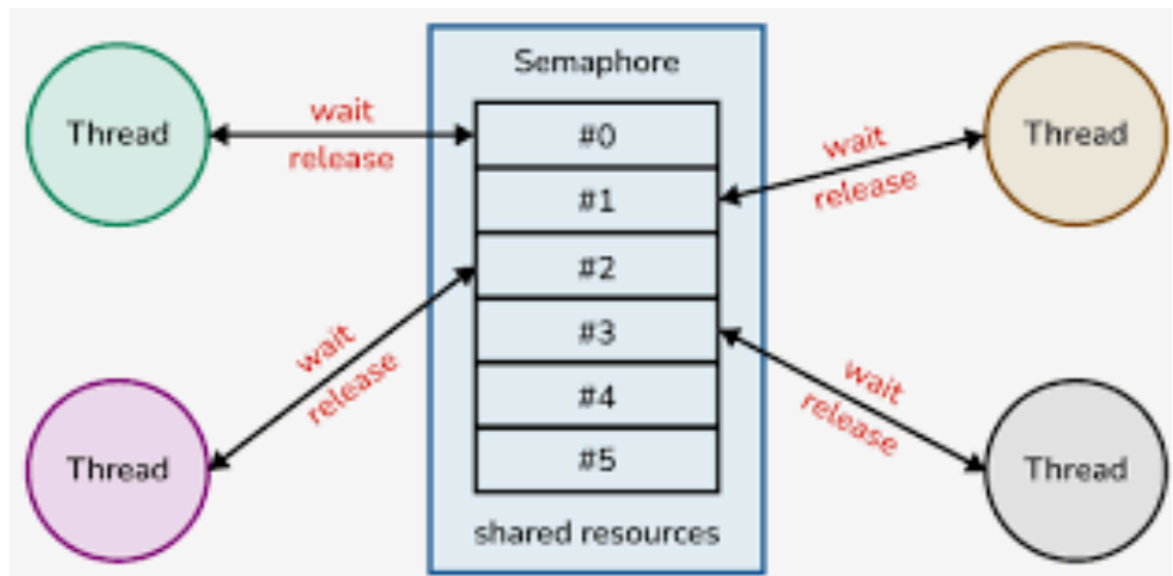
Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 SOFTWARE ENGINEERING

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

1. Dasar Teori



Semaphore adalah primitif sinkronisasi utama dalam sistem operasi Xinu yang digunakan untuk mengatur akses ke sumber daya bersama dan mengoordinasikan proses dalam lingkungan multitasking. Beroperasi menggunakan variabel integer (counting semaphore), mekanisme ini mencatat jumlah kapasitas sumber daya yang tersedia di dalam sistem secara berkala. Melalui pendekatan ini, kernel dapat memastikan bahwa interaksi antarproses berjalan dengan aman tanpa menyebabkan konflik perebutan data.

Pengelolaan nilai integer tersebut dikendalikan secara mutlak oleh dua operasi atomik, yaitu `wait()` dan `signal()`. Operasi `wait()` berfungsi untuk mengambil sumber daya dengan mengurangi nilai hitungan semaphore, di mana proses akan langsung diblokir ke status WAIT jika nilai hitungan berubah menjadi negatif. Sebaliknya, operasi `signal()` berperan melepaskan sumber daya dengan menambah hitungan semaphore sekaligus membangunkan proses yang tertahan di antrean tunggu untuk dipindahkan kembali ke daftar siap (READY list).

2. Manfaat

- Memahami Sinkronisasi Proses Tingkat Rendah: Saya dapat mendalami secara langsung bagaimana kernel sistem operasi (seperti Xinu) menggunakan variabel integer dan antrean (queue) untuk mengatur koordinasi antarproses yang berjalan bersamaan.

```
#include <xinu.h>

void p1(sid32 s1, sid32 s2);
void p2(sid32 s2, sid32 s3);
void p3(sid32 s3, sid32 s1);

process main(void)
{
    sid32 s1, s2, s3;

    s1 = semcreate(1);
    s2 = semcreate(0);
    s3 = semcreate(0);

    resume(create(p1, 1024, 20, "P1", 2, s1, s2));
    resume(create(p2, 1024, 20, "P2", 2, s2, s3));
    resume(create(p3, 1024, 20, "P3", 2, s3, s1));

    return OK;
}

void p1(sid32 s1, sid32 s2)
{
    while(1)
    {
        wait(s1);
        printf("Kwak\n");
    }
}
```

- B. [50 poin] Buatlah proses bernama produser yang memproduksi bilangan 1-1000. Buatlah proses bernama konsumen yang akan menampilkan nilai yang diproduksi oleh produser. Gunakan semaphore!

Produser memproduksi nilai 1

Konsumer menampilkan nilai 1

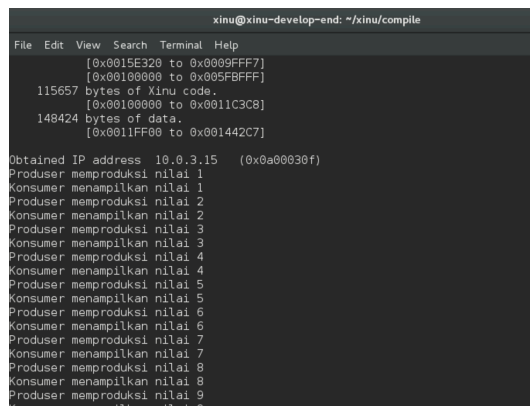
Produser memproduksi nilai 2

Konsumer menampilkan nilai 2

Produser memproduksi nilai 1000 Konsumer menampilkan nilai 1000



```
Open [R] *main.c -binu/system Save [X]  
#include <xinu.h>  
void produser(sid32 empty, sid32 full);  
void konsumer(sid32 full, sid32 empty);  
  
int 32 buffer = 0;  
process main(void)  
{  
    sid32 empty, full;  
    empty = semcreate(1);  
    full = semcreate(0);  
    resume(create(produser, 1024, 20, "produser", 2, empty, full));  
    resume(create(konsumer, 1024, 20, "konsumer", 2, full, empty));  
    return OK;  
}  
  
void produser(sid32 empty, sid32 full)  
{  
    int32 i;  
    for(i = 1; i <= 1000; i++)  
    {  
        wait(empty);  
        buffer = i;  
        signal(full);  
    }  
}
```



```
xinu@xinu-develop-end: ~/xinu/compile  
File Edit View Search Terminal Help  
[0x0015E320 to 0x0009FFF7]  
[0x00100000 to 0x005FBFFF]  
115657 bytes of Xinu code.  
[0x00100000 to 0x0011C3C8]  
148424 bytes of data.  
[0x0011FF00 to 0x001442C7]  
  
Obtained IP address 10.0.3.15 (0xa00030f)  
Produser memproduksi nilai 1  
Konsumer menampilkan nilai 1  
Produser memproduksi nilai 2  
Konsumer menampilkan nilai 2  
Produser memproduksi nilai 3  
Konsumer menampilkan nilai 3  
Produser memproduksi nilai 4  
Konsumer menampilkan nilai 4  
Produser memproduksi nilai 5  
Konsumer menampilkan nilai 5  
Produser memproduksi nilai 6  
Konsumer menampilkan nilai 6  
Produser memproduksi nilai 7  
Konsumer menampilkan nilai 7  
Produser memproduksi nilai 8  
Konsumer menampilkan nilai 8  
Produser memproduksi nilai 9  
Konsumer menampilkan nilai 9
```

4. Kesimpulan

saya dapat menyimpulkan bahwa saya telah berhasil memahami peran vital semaphore sebagai primitif sinkronisasi utama dalam sistem operasi Xinu. Melalui pemahaman konsep variabel integer ini, saya mengetahui bagaimana kernel mengoordinasikan proses multitasking secara aman dan mencegah terjadinya konflik perebutan data saat mengakses sumber daya bersama.

Selain itu, saya juga berhasil mendalami mekanisme kerja dari dua operasi atomik utama, yaitu wait dan signal, yang berfungsi untuk memblokir proses ke status WAIT maupun membangunkan proses kembali ke READY list. Pengalaman praktis dalam mengimplementasikan dan mengamati alur kerja semaphore ini tidak hanya memperkuat pemahaman teoretis saya mengenai sinkronisasi tingkat rendah, tetapi juga meningkatkan keterampilan saya dalam merancang pemrograman konkuren yang efisien sekaligus menghindari risiko race condition di dalam sistem.